

HW Assignment 1: Apache Spark

Name: Olorunfunmi Shobowale

Date: September 28, 2026

Part 1: Implementing Apache Spark and an Example Problem (40 points)

1.1 Background

Apache Spark is an open source engine for processing large amounts of data across many machines. The main reason it became popular is that it keeps intermediate results in memory instead of writing them to disk between every step, which is what Hadoop MapReduce does. This makes Spark much faster for jobs that pass over the same data many times, like machine learning or interactive queries (Zaharia et al., 2016).

A Spark program has a driver, which is the main program I write. The driver breaks the work into tasks and sends them to executors through a cluster manager. On a real cluster the executors run on different machines, but in local mode, which is what I used, everything runs on one computer and the executors are just threads. The nice part is that the same code works in both cases (Apache Software Foundation, 2026a).

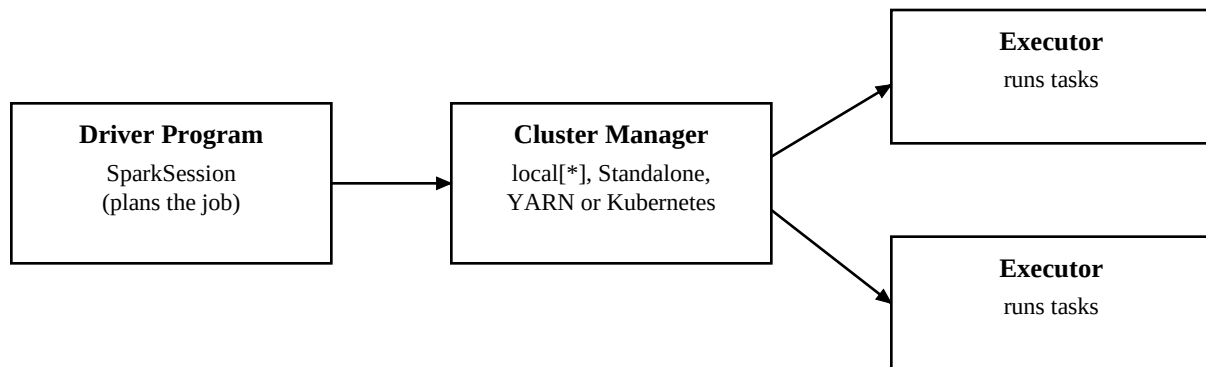


Figure 1. Basic parts of a Spark application. Redrawn from the Spark "Cluster Mode Overview" page (Apache Software Foundation, 2026b).

1.2 My computing environment

Table 1 shows the machine and software versions I used for this assignment.

Component	Version / setting
Operating system	Ubuntu 24.04.4 LTS (64-bit Linux)
Hardware	Intel Xeon @ 2.10 GHz, 4 CPU cores, 16 GB RAM
Java (JDK)	OpenJDK 21.0.10
Python	3.11.15 (in a virtual environment)
Apache Spark / PySpark	4.2.0, installed with pip
Deployment mode	Local mode, master = local[*]

Table 1. Environment used for this assignment.

1.3 How I installed Spark

Java was already installed on the machine, so I only had to check the version. Spark 4 needs Java 17 or 21, and I had 21. After that I made a virtual environment for Python and installed PySpark into it with pip. The PySpark package from pip already contains the whole Spark engine, so I did not need to download Spark separately for local mode. These are the commands I ran:

```
$ java -version
openjdk version "21.0.10" 2026-01-20

$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv)$ pip install --upgrade pip setuptools wheel
(.venv)$ pip install pyspark
(.venv)$ python -c "import pyspark; print(pyspark.__version__)"
4.2.0
```

The install itself went through without errors. The problem I ran into came the first time I ran a Spark program. Before printing any results, Spark printed a few warnings, and one of them was this:

```
WARN Utils: Your hostname, vm, resolves to a loopback address: 127.0.0.1;
            using 192.0.2.2 instead (on interface eth0)
WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
```

This happens because the machine name points to 127.0.0.1, so Spark is not sure which network address to use and picks one on its own. In local mode the program still ran fine, but on a real cluster the wrong address can stop the executors from reaching the driver. The warning itself tells you the fix, which is to set the `SPARK_LOCAL_IP` environment variable. I ran the program again with it set and the warning was gone:

```
(.venv)$ export SPARK_LOCAL_IP=127.0.0.1
(.venv)$ python part1_wordcount.py
```

There was also a warning that says "Unable to load native-hadoop library for your platform". This one is normal in local mode. Spark just uses its built in Java code instead, so I left it alone. To keep the output readable I also set the log level to ERROR inside my scripts with `setLogLevel("ERROR")`.

1.4 Example problem: word count

To test the installation I used the word count problem. The idea is simple. Given a text file, count how many times each word shows up and list the most common ones. It is used a lot as a first Spark program because it uses the same map, shuffle and reduce steps that bigger jobs use. My input file `data.txt` has five sentences about Spark.

Code (`part1_wordcount.py`):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("COS611-WordCount").master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
print("Spark version:", spark.version)
print("Master:", spark.sparkContext.master)
print("Default parallelism:", spark.sparkContext.defaultParallelism)

lines = spark.sparkContext.textFile("data.txt")
counts = (lines.flatMap(lambda line: line.lower().split())
          .map(lambda word: (word, 1))
          .reduceByKey(lambda a, b: a + b)
          .sortBy(lambda kv: -kv[1]))
print("Total lines:", lines.count())
print("Top 8 words:")
for word, n in counts.take(8):
    print(f" {word:<12}{n}")
spark.stop()
```

Here is what each part does. `textFile` reads the file into an RDD where each element is one line. `flatMap` splits every line into words and puts them all in one list. `map` turns each word into a pair like `("spark", 1)`. `reduceByKey` adds up the numbers for each word, and this is the step where Spark has to move data between partitions (a shuffle). Last, `sortBy` puts the words in order from most to least common. One thing I found interesting is that none of this actually runs until `count()` or `take()` is called. Spark waits and builds a plan first, which is called lazy evaluation.

Output:

```
(.venv)$ python part1_wordcount.py
Spark version: 4.2.0
Master: local[*]
Default parallelism: 4
Total lines: 5
Top 8 words:
  spark      6
  data       4
  analytics  3
  in         3
  for        2
  big        2
  is         2
  apache     1
```

The output shows that Spark is working. It started in local mode, it used all 4 cores (default parallelism is 4), it read all 5 lines, and the counts are correct. I checked "spark" by hand and it does appear 6 times in the file.

Part 2: Tutorial on Setting Up Spark and Running Basic Operations (50 points)

This part is a step by step guide for someone starting from a normal computer with nothing installed. I wrote the commands for Linux and macOS and added notes where Windows is different.

Step 1: Check the system requirements

You need a 64-bit computer with at least 4 GB of RAM, but 8 GB or more is better. You also need about 1 GB of free disk space. Spark runs on Java, and Spark 4 works with Java 17 or Java 21 (Apache Software Foundation, 2026a). If you want to write Spark programs in Python, you also need Python 3.10 or newer.

Step 2: Install Java

```
# Ubuntu / Debian
$ sudo apt update
$ sudo apt install -y openjdk-21-jdk

# macOS with Homebrew
$ brew install openjdk@21

# Windows: download and install Eclipse Temurin JDK 21 from adoptium.net

# Check it on any system
$ java -version
```

If Spark later says it cannot find Java, set the `JAVA_HOME` variable to the folder where the JDK is installed. On Linux that looks like `export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64`. On Windows it is set under System Properties, Environment Variables.

Step 3: Set up Python and a virtual environment

I recommend a virtual environment so that Spark and its packages stay separate from everything else on the computer.

```
$ python3 --version          # should be 3.10 or newer
$ python3 -m venv .venv      # Windows: python -m venv .venv
$ source .venv/bin/activate  # Windows: .venv\Scripts\activate
(.venv)$ pip install --upgrade pip setuptools wheel
```

Step 4: Install Apache Spark

There are two ways to do this. The first way is the one I used. You install PySpark with pip, and it comes with the full Spark engine, which is enough for running Spark on one computer (Apache Software Foundation, 2026c).

```
(.venv)$ pip install pyspark
```

The second way is to download the full Spark package from spark.apache.org/downloads, unzip it, and add it to your PATH. This also gives you `spark-shell` for Scala and `spark-submit` for sending jobs to a cluster.

```
$ tar -xzf spark-4.x.x-bin-hadoop3.tgz
$ export SPARK_HOME=~/.spark-4.x.x-bin-hadoop3
$ export PATH=$SPARK_HOME/bin:$PATH
```

Windows users need one extra thing. Spark on Windows needs a file called `winutils.exe`. Put it in `C:\hadoop\bin` and set `HADOOP_HOME=C:\hadoop`. If you skip this, saving files from Spark will fail with an error about `HADOOP_HOME` not being set.

Step 5: Check that Spark works

```
(.venv)$ python -c "import pyspark; print(pyspark.__version__)"
4.2.0
(.venv)$ pyspark          # opens the Spark shell, type exit() to leave
```

If the version number prints and the shell opens with the Spark logo, the setup is done. As I mentioned in Part 1, if you see the loopback address warning, set `SPARK_LOCAL_IP=127.0.0.1`.

Step 6: Make some sample data

I made two small CSV files so the results would be easy to check by hand. `sales.csv` has 10 orders and `regions.csv` has the manager for each region.

```
sales.csv
order_id,region,product,quantity,unit_price
1,East,Laptop,2,950.00
2,West,Phone,5,610.00
3,East,Tablet,3,320.00
4,North,Laptop,1,980.00
5,South,Phone,4,590.00
6,West,Laptop,2,940.00
7,North,Tablet,6,300.00
8,East,Phone,3,620.00
9,South,Tablet,2,310.00
10,West,Tablet,4,315.00

regions.csv
region,manager
East,A. Johnson
West,B. Okafor
North,C. Williams
South,D. Adeyemi
```

Step 7: Run basic Spark operations

The script below, `part2_operations.py`, does seven basic operations. The first one uses RDDs, which is the older low level API, and the rest use DataFrames and Spark SQL, which is what most people use now. I ran it with `python part2_operations.py` and the output of each operation is shown after the code.

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("COSC611-BasicOps").master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

print("=== Op 1: Create an RDD and apply map / filter / reduce ===")
rdd = spark.sparkContext.parallelize(range(1, 11))
squares = rdd.map(lambda x: x * x)
evens = squares.filter(lambda x: x % 2 == 0)
print("squares:", squares.collect())
print("even squares:", evens.collect())
print("sum of squares:", squares.reduce(lambda a, b: a + b))

print("\n=== Op 2: Load a CSV file into a DataFrame ===")
df = spark.read.csv("sales.csv", header=True, inferSchema=True)
df.printSchema()
df.show(5)

print("=== Op 3: Select, add a column, and filter ===")
df2 = df.withColumn("revenue", F.col("quantity") * F.col("unit_price"))
df2.select("order_id", "product", "revenue").filter(F.col("revenue") > 1500).show()

print("=== Op 4: Group by and aggregate ===")
(df2.groupBy("region")
 .agg(F.sum("revenue").alias("total_revenue"), F.count("*").alias("orders"))
 .orderBy(F.desc("total_revenue"))
 .show())

print("=== Op 5: Join two DataFrames ===")
regions = spark.read.csv("regions.csv", header=True, inferSchema=True)
(df2.join(regions, on="region", how="inner")
 .select("order_id", "region", "manager", "revenue")
 .orderBy("order_id")
 .show())

print("=== Op 6: Spark SQL query ===")
df2.createOrReplaceTempView("sales")
spark.sql("""
    SELECT product, SUM(quantity) AS units, ROUND(AVG(unit_price),2) AS avg_price
    FROM sales GROUP BY product ORDER BY units DESC
    """).show()

print("=== Op 7: Write results to disk (Parquet) and read back ===")
df2.write.mode("overwrite").parquet("output/sales_parquet")
back = spark.read.parquet("output/sales_parquet")
print("rows read back from Parquet:", back.count())
spark.stop()
```

Operation 1: RDD map, filter and reduce

`parallelize` takes a normal Python list of the numbers 1 to 10 and spreads it out as an RDD. `map` squares each number and `filter` keeps only the even squares. These two are transformations, so they do not run right away. `collect` and `reduce` are actions and they make Spark do the work. I checked the sum by hand: $1 + 4 + 9 + \dots + 100 = 385$, which matches.

```
squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
even squares: [4, 16, 36, 64, 100]
sum of squares: 385
```

Operation 2: Load a CSV file into a DataFrame

Because I used `inferSchema=True`, Spark looked at the data and figured out the column types by itself (integer, string and double). `printSchema` shows the types and `show(5)` prints the first 5 rows.

```
root
 |-- order_id: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- product: string (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- unit_price: double (nullable = true)

+-----+-----+-----+-----+-----+
|order_id|region|product|quantity|unit_price|
+-----+-----+-----+-----+-----+
|      1|  East| Laptop|       2|    950.0|
|      2|  West|  Phone|       5|    610.0|
|      3|  East| Tablet|       3|    320.0|
|      4|North| Laptop|       1|    980.0|
|      5|South|  Phone|       4|    590.0|
+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Operation 3: Add a column and filter rows

`withColumn` makes a new column called `revenue`, which is `quantity` times `unit_price`. Then `select` keeps only three columns and `filter` keeps orders with revenue over \$1,500. Six of the ten orders passed.

```
+-----+-----+-----+
|order_id|product|revenue|
+-----+-----+-----+
|      1| Laptop| 1900.0|
|      2|  Phone| 3050.0|
|      5|  Phone| 2360.0|
|      6| Laptop| 1880.0|
|      7| Tablet| 1800.0|
|      8|  Phone| 1860.0|
+-----+-----+-----+
```

Operation 4: Group by and aggregate

groupBy with agg gives the total revenue and number of orders for each region. West is first with \$6,190 from 3 orders. Checking by hand: $3050 + 1880 + 1260 = 6190$.

```
+-----+-----+-----+
|region|total_revenue|orders|
+-----+-----+-----+
|  West|      6190.0|     3|
|  East|      4720.0|     3|
| South|      2980.0|     2|
| North|      2780.0|     2|
+-----+-----+-----+
```

Operation 5: Join two DataFrames

This joins the sales data with the regions file on the region column, so each order now shows its manager. It works the same way as an inner join in SQL.

```
+-----+-----+-----+-----+
|order_id|region|  manager|revenue|
+-----+-----+-----+-----+
|      1|  East| A. Johnson| 1900.0|
|      2|  West| B. Okafor| 3050.0|
|      3|  East| A. Johnson|   960.0|
|      4| North| C. Williams|   980.0|
|      5| South| D. Adeyemi| 2360.0|
|      6|  West| B. Okafor| 1880.0|
|      7| North| C. Williams| 1800.0|
|      8|  East| A. Johnson| 1860.0|
|      9| South| D. Adeyemi|   620.0|
|     10|  West| B. Okafor| 1260.0|
+-----+-----+-----+-----+
```

Operation 6: Spark SQL query

createOrReplaceTempView registers the DataFrame as a table called sales, so I can query it with normal SQL. This query finds total units sold and average price for each product. Behind the scenes Spark turns SQL and DataFrame code into the same kind of plan (Apache Software Foundation, 2026d).

```
+-----+-----+-----+
|product|units|avg_price|
+-----+-----+-----+
| Tablet|   15|   311.25|
|  Phone|   12|   606.67|
| Laptop|    5|   956.67|
+-----+-----+-----+
```

Operation 7: Save to Parquet and read it back

The last step saves the data as Parquet, which is a compressed column based file format that is common in big data. Then it reads the file back to make sure nothing was lost, and all 10 rows came back. When I looked in the output folder, Spark had written a folder instead of one file. Inside it were the data file and an empty file called `_SUCCESS` that shows the write finished.

```
rows read back from Parquet: 10
```

Step 8: Watch the job and shut down

While a Spark program is running you can open <http://localhost:4040> in a browser. This is the Spark web UI, and it shows the jobs, stages and tasks, how long each one took, and how much memory is being used. It is the main place to look when a program is slow. At the end of every program you should call `spark.stop()` so that Spark gives back the memory and CPU it was using.

Summary of the operations

#	Operation	API	Methods used
1	Map, filter and reduce	RDD	parallelize, map, filter, reduce
2	Load CSV data	DataFrame	read.csv, printSchema, show
3	New column and filter	DataFrame	withColumn, select, filter
4	Group and aggregate	DataFrame	groupBy, agg, sum, count
5	Join	DataFrame	join
6	SQL query	Spark SQL	createOrReplaceTempView, sql
7	Save and reload	I/O	write.parquet, read.parquet

Table 2. The seven operations in the tutorial.

Part 3: Challenges and Limitations of Apache Spark (10 points)

Getting Spark to run on one machine was not very hard, but while doing this assignment and reading about it I noticed several problems that would matter more on a real project.

1. Spark needs a lot of memory. Its speed comes from keeping data in RAM, so RAM becomes the main cost. If the data does not fit in memory, Spark has to spill to disk, which makes it slow, or the job fails with an out of memory error. Picking the right memory and partition settings takes practice (Karau & Warren, 2017).
2. Versions have to match. Spark depends on the right versions of Java, Python or Scala, and the Hadoop libraries. Spark 4 does not run on Java 8 or 11 anymore. On Windows there is the extra winutils.exe and HADOOP_HOME step, which a lot of beginners get stuck on.
3. Networking can cause problems. I saw this myself with the loopback address warning in Part 1. On one computer it was only a warning, but on a cluster the driver and executors must be able to reach each other, so the network has to be set up correctly.
4. Errors are hard to track down. Because of lazy evaluation, a mistake in one line might not show up until an action runs much later. The error messages from PySpark also mix Python and Java stack traces, which makes them long and confusing.
5. Shuffles are expensive. Operations like groupBy, join and reduceByKey have to move data between machines over the network. On big data this is often the slowest part of the job. If one key has much more data than the others (data skew), one task ends up doing most of the work.
6. It is not the best choice for everything. For small data, Spark takes a few seconds just to start, so plain Python with pandas is often faster. For streaming, Spark works in small batches by default, so the delay is higher than tools built for handling one record at a time, like Apache Flink.
7. Spark does not store data. It only does the processing, so it has to be used together with something like HDFS, Amazon S3 or a database. That means more systems to install, pay for and secure.
8. Python UDFs are slow. When you write your own Python function and use it on a DataFrame, the data has to go back and forth between Java and Python. The built in Spark functions are much faster, so you have to learn to use them instead.
9. Clusters cost money and need skills. Running Spark in production, on your own servers or on a cloud service like Databricks, Amazon EMR or Google Dataproc, can be expensive. The team also needs to know about distributed systems, tuning and security, since things like authentication and encryption are not turned on by default.

Overall, Spark was easy to set up in local mode and it handled both the RDD and the DataFrame and SQL examples correctly. Most of its real difficulties show up at a larger scale, with memory, shuffles, tuning and running a cluster. Knowing these limits helps in deciding when Spark is worth using and when a simpler tool is good enough.

References

- Apache Software Foundation. (2026a). *Apache Spark documentation: Overview*.
<https://spark.apache.org/docs/latest/>
- Apache Software Foundation. (2026b). *Cluster mode overview*.
<https://spark.apache.org/docs/latest/cluster-overview.html>
- Apache Software Foundation. (2026c). *PySpark installation*.
https://spark.apache.org/docs/latest/api/python/getting_started/install.html
- Apache Software Foundation. (2026d). *Spark SQL, DataFrames and Datasets guide*.
<https://spark.apache.org/docs/latest/sql-programming-guide.html>
- Karau, H., & Warren, R. (2017). *High performance Spark: Best practices for scaling and optimizing Apache Spark*. O'Reilly Media.
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56-65. <https://doi.org/10.1145/2934664>